Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO) Algorithms: Reproduction and Comparison Study

Ali Asad, Adam Bayley, Amyn Sorkhilalehloo, Reza Jahantigh, Amir Mohammad Ramezan Naderi
Queen's University
{24xt20, 19ahb, 24fl4, 22sy14, 24fbdg}@queensu.ca

Abstract

Reinforcement learning (RL) algorithms such as Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO) are designed to achieve stable and efficient policy training. This project implements both algorithms from scratch and evaluates their performance in the Atari `Breakout-v5` environment from the Gymnasium suite. We compare TRPO and PPO based on sample efficiency, training stability, and computational cost. While environment interaction relies on existing libraries, all core learning components are independently developed. Our controlled experiments aim to characterize the trade-offs between these two policy optimization methods. We find that, despite TRPO's mathematical guarantees, PPO outperforms TRPO in all metrics.

1. Introduction

RL has demonstrated high levels of success in solving complex decision-making tasks, particularly through policy optimization techniques. Recent advancements in policy optimization techniques, such as DeepSeek's Gradient-based Reinforcement Policy Optimization (GRPO) method, have become increasingly prominent due to their ability to directly optimize stochastic policies [10]. As RL continues to advance, the need for stable and efficient policy learning methods has become increasingly important. Among the most widely used policy gradient methods are TRPO and PPO, both introduced to improve the stability and efficiency of training reinforcement learning agents [3]. TRPO enforces a constraint on policy updates to prevent drastic changes, while PPO simplifies this approach with a clipped surrogate objective [7, 9]. These techniques have proven effective in various applications, from robotic control to game playing. In this project, we aim to implement these algorithms from scratch and evaluate their performance on standard benchmark environments.

Optimizing policies in reinforcement learning is challenging due to unstable updates, sample inefficiency, and sensitivity to hyperparameters. Small updates can slow learning, while large updates can destabilize training. Finding a balance between stability and efficiency is critical for practical applications. Many researchers have published work describing methods for finding optimal hyperparameter values, however, it remains a difficult task to properly tune policy gradient methods [3].

Although TRPO and PPO follow similar overall structures, they differ in how they constrain policy updates. TRPO enforces a strict KL-divergence constraint to ensure stable and monotonic policy improvement. In contrast, PPO uses a clipped surrogate objective that heuristically limits the update size, allowing for greater flexibility and easier implementation. Both methods aim to improve training stability and efficiency but differ in their trade-offs between computational cost, robustness, and applicability.

This project aims to determine whether TRPO's theoretical guarantees are sufficient to ensure competitive performance relative to PPO. Specifically, we investigate:

- The ability of different policy update mechanisms to improve training stability.
- How sample efficiency varies between methods when applied to different types of tasks.
- The computational demands of TRPO and PPO when scaling to larger environments.

By conducting controlled experiments, we aim to determine which method offers better trade-offs between stability, sample efficiency, and computational cost in real-world applications.

2. Background

2.1. Reinforcement Learning

Reinforcement learning (RL) is a machine learning paradigm where an agent learns to maximize cumulative rewards through trial-and-error interactions with an environment [11]. Unlike supervised learning, RL operates under partial observability and delayed feedback, requiring agents to balance exploration (discovering new strategies)

and exploitation (leveraging known rewards). Key challenges include high-dimensional state spaces (e.g., raw pixels in Atari games) and sparse reward signals, which are addressed through advancements like deep neural networks for function approximation [5] and advanced credit assignment techniques [12]. The following sections formalize RL's mathematical framework and core algorithms.

2.1.1. Markov Decision Processes (MDPs)

Reinforcement learning (RL) formalizes sequential decision-making through Markov Decision Processes (MDPs), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{P}(s'|s,a)$ the transition dynamics, $\mathcal{R}(s,a)$ the reward function, and $\gamma \in [0,1)$ the discount factor [11]. The goal is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative reward $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

2.1.2. Policy Gradient Methods

Policy gradient methods optimize stochastic policies directly by ascending the gradient of the expected return with respect to policy parameters θ. The REINFORCE algorithm [14] provides an unbiased gradient estimate:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t \right],$$

where G_t is the return from time t. To reduce variance, modern approaches often use actor-critic architectures that combine policy gradients with value function approximation [12].

2.1.3. Actor-Critic Methods

Actor-Critic methods combine policy optimization, **the actor**, with value function approximation, **the critic**, to reduce the variance of policy gradient estimates [12]. The actor updates the policy $\pi_\theta(a|s)$ using gradient ascent, while the critic approximates the value function $V_\phi(s)$ or $Q_\phi(s,a)$ to evaluate actions. The gradient update takes the form:

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\nabla_\theta \log \pi_\theta(a|s) \hat{A}(s,a) \right],$$

where $\hat{A}(s,a)$ is the advantage estimate provided by the critic. These frameworks enable online learning and temporal difference (TD) updates or for modern implementations, often employ shared network architectures between the actor and critic to learn features jointly [6].

2.1.4. Advantage Estimation

Advantage functions $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ measure the relative benefit of taking action a in state s. Generalized Advantage Estimation (GAE) [8] provides a low-variance, biased estimator using exponentially weighted averages of λ-returns:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l},$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. GAE balances bias and variance through λ.

2.2. Trust Region Methods

Trust region methods stabilize policy optimization by restricting updates to a local region where approximations remain valid. These methods, rooted in constrained optimization theory [2], ensure policy improvements without catastrophic performance collapse—a common issue in naive policy gradient approaches. In RL, trust region methods are particularly critical for high-dimensional or continuous action spaces, where gradient steps can easily overshoot into poorly performing policies. The Trust Region Policy Optimization (TRPO) algorithm [7] formalizes this idea by enforcing a KL divergence constraint between successive policies, enabling monotonic policy improvement guarantees [4]. Proximal Policy Optimization (PPO) [9] later emerged as a simplified alternative, replacing TRPO's constrained optimization with clipped probability ratios to heuristically enforce trust regions. While PPO sacrifices theoretical guarantees for implementation ease, it retains the core trust region philosophy by limiting policy update magnitudes [3]. Both methods address the same fundamental challenge: balancing policy improvement with stability.

2.2.1. TRPO

Trust Region Policy Optimization (TRPO) [7] ensures stable policy updates by constraining the KL divergence between old and new policies. Inspired by conservative policy iteration [4], TRPO maximizes the surrogate objective:

$$\max_\theta \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right],$$

subject to $\mathbb{E}_t[\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)] \leq \delta$. This avoids catastrophic policy collapse by limiting update magnitudes [3].

2.2.2. PPO

Proximal Policy Optimization (PPO) [9] is a simplified variant of TRPO that replaces the trust-region constrained update used in TRPO with a clipped surrogate objective to restrict policy changes. PPO maximizes the surrogate objective:

$$\max_\theta \ \mathbb{E}_t \left[\min \left(r_t(\theta)\, \hat{A}_t,\ \text{clip}\left(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\right) \hat{A}_t \right) \right],$$

Here $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and ϵ is a small clipping threshold. By clipping $r_t(\theta)$ within $[1-\epsilon, , 1+\epsilon]$, PPO effectively constrains the policy update, achieving stable performance comparable to TRPO without the need for an explicit KL-divergence constraint [9].

3. Methodology

3.1. Environment Description (ALE Breakout-v5)

The Arcade Learning Environment (ALE) [1] provides a standardized framework for reinforcement learning (RL) experiments on Atari 2600 games. We use the `Breakout-v5` environment, where the agent controls a paddle to deflect a ball and break bricks. Observations consist of raw pixel frames (210×160 RGB). The episode terminates when all bricks are cleared or the agent exhausts its lives. There are four actions defined for this game: left, right, fire, no-op. Rewards are sparse, with +1 for each brick broken and 0 otherwise. To make observations suitable for deep reinforcement learning, we apply the following preprocessing pipeline adapted from Mnih et al. [5]:

- **Frame Skipping**: Repeat each action for 4 frames (skip $= 4$), reducing computation by a factor of 4 while maintaining temporal coherence.
- **Grayscale Conversion**: Convert RGB frames to single-channel grayscale, reducing input dimensionality.
- **Downsampling**: Resize frames to 84×84 pixels via area interpolation, preserving spatial structure while minimizing aliasing.
- **Normalization**: Scale pixel values from [0, 255] to [0, 1] to stabilize neural network training.
- **Frame Stacking**: Maintain a deque of the last 4 processed frames (stack $= 4$) and concatenate them along the channel dimension, yielding a final observation tensor of shape $(84, 84, 4)$.

3.2. Network Architecture

3.2.1. Policy Network

The policy network follows a convolutional neural network (CNN) architecture adapted from [5] and [7]. Input frames pass through three convolutional layers (32 8×8, 64 4×4, and 64 3×3 filters with strides 4, 2, and 1, respectively), followed by a fully connected layer (512 units). The output layer uses a softmax activation to parameterize a categorical distribution over actions. ReLU activations are applied after all hidden layers.

3.2.2. Value Network

The value network shares the same CNN backbone as the policy network but terminates in a linear output layer to estimate the state value $V(s)$. This design enables feature sharing between the policy and value functions while maintaining separate output heads to prevent gradient conflicts [7].

3.3. TRPO Implementation

3.3.1. Surrogate Loss Formulation

TRPO maximizes the expected improvement via the surrogate objective:

$$L_{\text{surr}}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right],$$

where $\hat{A}_t$ denotes the advantage estimate computed using generalized advantage estimation (GAE) [8].

3.3.2. Conjugate Gradient

To efficiently compute the natural gradient direction $F^{-1}g$ (where F is the Fisher information matrix and g the policy gradient), we use the conjugate gradient method with automatic tolerance adjustment [7]. This avoids the cubic computational complexity of explicitly inverting F.

3.3.3. Fisher-Vector Products

Fisher-vector products Fv are approximated using the identity $Fv = \nabla_\theta \left(\nabla_\theta \log \pi_\theta(a|s) \cdot v \right)$, enabling efficient curvature estimation without storing F explicitly [7].

3.3.4. KL Divergence Constraint

Policy updates are constrained by a KL divergence threshold $\delta = 0.01$:

$$\mathbb{E}_t \left[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_\theta(\cdot|s_t)] \right] \leq \delta.$$

This ensures stable policy updates while avoiding catastrophic forgetting.

3.3.5. Line Search

After computing the candidate update direction, a backtracking line search iteratively reduces the step size until the KL constraint and surrogate loss improvement are satisfied [7].

3.4. PPO Implementation

3.4.1. Surrogate Loss and Clipping

Proximal Policy Optimization (PPO) maximizes a clipped surrogate objective that heuristically constrains policy updates without requiring second-order optimization. The clipped objective is defined as:

$$\mathcal{L}_{clip}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\, \hat{A}_t,\ \text{clip}\left(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\right) \hat{A}_t \right) \right]$$

3.4.2. value function loss

The value network V_θ is optimized using the mean squared error (MSE) loss:

$$\mathcal{L}_{critic}(\theta) = \mathbb{E}_t \left[\left(V_\theta(s_t) - \hat{R}_t \right)^2 \right],$$

where $\hat{R}_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ is the Monte Carlo return.

3.4.3. Actor-Critic Architecture and Optimization

PPO agent employs an actor-critic model using a shared convolutional backbone followed by two heads: one for the action logits (actor) and one for the value estimate (critic). The policy outputs a categorical distribution over discrete actions.

The total loss function combines the clipped surrogate objective, value function loss, and an entropy regularization term:

$$\mathcal{L}_{total}(\theta) = \mathcal{L}_{clip}(\theta) - C_2\,\mathbb{E}_t\left[\mathcal{H}\left[\pi_\theta(\cdot|s_t)\right]\right] + C_1\mathcal{L}_{critic}(\theta)$$

where

$\mathcal{H}$ denotes entropy, encouraging exploration. Optimization is performed using the Adam optimizer with a learning rate of $1e^{-4}$, over minibatches sampled from collected trajectories

4. Experimental Setup

4.1. Training Protocol

The training loop for both TRPO and PPO follows four main stages per epoch. While the core stages remain largely similar, the two methods differ in their policy update step, as outlined below.

1. **Rollout Collection**: For each epoch, the agent interacts with the environment using the current policy to collect the required number of timesteps. Observations are preprocessed into stacked $84\times84\times4$ tensors using a custom frame-stacking wrapper.
2. **Advantage Computation**: Advantages $\hat{A}_t$ are estimated using Generalized Advantage Estimation (GAE) [8] as follows:

$$\hat{A}_t = \sum_{l=0}^{T-t}(\gamma\lambda)^l\delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

   where $\gamma = 0.99$ and $\lambda = 0.95$. The advantages are normalized across the batch to stabilize training.
3. **Critic Update**: The value network V_ϕ is optimized using the mean squared error (MSE) loss:

$$\mathcal{L}_\phi = \mathbb{E}_t\left[\left(V_\phi(s_t) - \hat{R}_t\right)^2\right],$$

   where $\hat{R}_t = \sum_{l=0}^{T-t}\gamma^l r_{t+l}$ is the Monte Carlo return.
4. **Policy Update**:

- **TRPO**: The policy π_θ is updated via constrained optimization:

$$\max_\theta\ \mathbb{E}_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\,\hat{A}_t\right] - \beta\,\mathcal{H}(\pi_\theta)$$

  subject to

$$\mathbb{E}_t\Big[\mathrm{KL}\Big(\pi_{\theta_{\text{old}}} \parallel \pi_\theta\Big)\Big] \le 0.01.$$

  The update direction is computed using 10 conjugate gradient iterations [7], followed by a backtracking line search.
- **PPO**: The policy is updated by maximizing the clipped surrogate objective:

$$\max_\theta\ \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}\Big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\Big)\hat{A}_t\Big)\Big]$$

  where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

  and ϵ is a small constant (e.g., 0.2). This clipping mechanism heuristically constrains the policy update [9].

4.2. Evaluation Metrics

We evaluate TRPO and PPO using metrics grouped into three categories:

- **Performance**:
  - *Mean Episode Reward*: Average undiscounted return per episode.
  - *Mean Episode Length*: Average steps per episode, indicating exploration efficiency.
- **Training Dynamics**:
  - *Sample Efficiency*: Reward per environment step $(\text{mean\_reward}/\text{total\_steps})$.
  - *Convergence Speed*: Epochs or steps to reach a reward threshold (e.g., 100 points in Breakout).
  - *KL Divergence*: Symmetric KL between old and new policies $(\mathrm{KL}(\pi_{\text{old}} \parallel \pi_{\text{new}}) + \mathrm{KL}(\pi_{\text{new}} \parallel \pi_{\text{old}}))$.
  - *Policy Entropy*: $\mathcal{H}(\pi(a|s))$, measuring exploration.
- **Computational Efficiency**:
  - *Wall-Clock Time per Epoch*: Seconds per training iteration.
  - *Training Stability*: Coefficient of variation $(\text{std\_reward}/\text{mean\_reward})$ across 5 seeds.

These metrics are logged at each epoch using the provided code, which computes KL divergence and entropy during policy updates, while reward statistics are derived from trajectory rollouts. Convergence speed is determined by the first epoch where the mean reward exceeds a predefined threshold.

4.3. Other Implementation Details

- **TimeLimit Wrapper**: Episodes terminate after 5,000 frames (1,250 agent steps) to prevent infinite episodes, implemented via Gymnasium's `TimeLimit` [13].
- **Logging**: Metrics (mean reward, KL divergence, entropy, etc.) are logged to CSV and stdout at each epoch. Reproducibility is ensured by fixing environment and PyTorch seeds across runs.

5. Results

In this section, we present the results of our experiments with the Proximal Policy Optimization (PPO) and Trust Region Policy Optimization (TRPO) algorithms in the Breakout-v5 environment. We explore the performance of both algorithms under varying hyperparameter configurations and compare their effectiveness based on key metrics such as average reward, episode length, entropy, and sample efficiency.

5.1. PPO results

We trained the PPO agent using three different hyperparameter settings. The model was trained for 5000 iterations for each configuration. The tested hyperparameter configurations are presented in Table 1.

Run	ϵ	C_1	C_2
Run 1	0.3	0.75	0.02
Run 2	0.1	0.75	0.1
Run 3	0.3	0.25	0.02

Table 1. Hyperparameter configurations tested during training. Here, ϵ represents the clipping threshold, C_1 denotes the value loss coefficient, and C_2 corresponds to the entropy regularization coefficient.

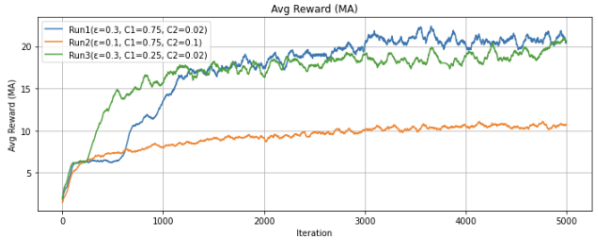
The training performance of each PPO configuration is visualized in Figure 1, showing metrics such as average reward, episode length, entropy, and reward per step.

5.1.1. Average Reward and Episode Length

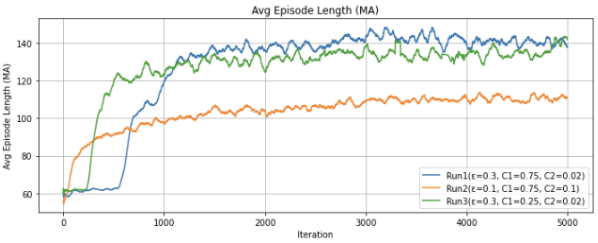
As shown in Figures 1(a) and (b), Run 1 achieved the highest average episodic reward and sustained it over time, followed closely by Run 3. In contrast, Run 2 consistently underperformed, exhibiting lower rewards and shorter episodes. In Run 1, the agent reached a reward of approximately 30 and an episode length of 150, which corresponds to human-level performance in Breakout. The low performance in Run 2 can be attributed to two factors:

1. **Smaller Clipping Range ($\epsilon = 0.1$):** A lower ϵ leads to overly conservative updates. This prevents the policy from making substantial improvements, potentially slowing convergence.
2. **High Entropy Coefficient ($C_2 = 0.1$):** Encourages excessive exploration by penalizing certainty in the policy distribution. As a result, the agent remains indecisive, which manifests in lower reward and shorter episodes.

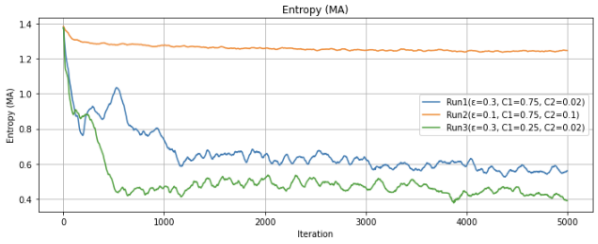
Run 3, despite reducing the value loss coefficient (C_1) from 0.75 to 0.25, showed similar performance to Run 1, suggesting that the PPO agent is robust to the specific choice of C_1. The main trade-off observed was a slightly noisier episode length curve in Run 3, likely due to under-optimized value estimates.



(a) Average Reward



(b) Average Episode Length



(c) Entropy

(d) Reward per Step

Figure 1. Training metrics for PPO under different hyperparameter configurations. Each run varies ϵ, C_1, and C_2 to examine performance sensitivity.

5.1.2. Entropy

Run 2 maintained a significantly higher entropy throughout training, as expected from the large C_2 term. This indicates the agent continued exploring suboptimal policies. Meanwhile, Run 1 and Run 3 showed a steady decline in entropy, a common indicator that the policy is becoming more deterministic and exploiting learned behaviors. We learned that the good value of C_2 is between 0.01 to 0.03.

5.1.3. Sample Efficiency

Reward per environment step is a direct measure of sample efficiency (It's reward over episode length). Both Run 1 and Run 3 exhibited efficient learning curves that steadily

increased and stabilized around 0.14. Run 2 lagged behind, remaining below 0.10 throughout training, further confirming the negative impact of overly cautious updates and excessive entropy regularization.

5.2. TRPO Results

We trained the TRPO agent in the same environment as the PPO agent, testing multiple hyperparameter configurations using a pseudo grid search. The full set of configurations explored during the grid search is extensive (as showen in Table 2), but for demonstration purposes and further analysis, Table 3 shows a selected sample of the hyperparameter combinations. In this table, C1 represents the number of CG iterations, C2 denotes the CG damping factor, and C3 refers to the entropy coefficient.

Hyperparameter	Values
`Gamma` (γ)	0.98, 0.99
`Lambda` (λ)	0.95, 0.97, 0.99
`Delta` (δ)	0.01, 0.1, 0.5, 1
`CG Iters`	5, 15, 20
`CG Damping`	0.05, 0.1
`Epochs`	50, 100, 150
`Steps per Epoch`	10, 20, 50, 100, 500
`Entropy Coeff`	0.1, 0.01, 0.05, 0.2, 0.5
`Reward Threshold`	3.0
`Entropy Decay Rate`	0.99, 0.95, 0.9

Table 2. Hyperparameter Combinations for Grid Search

Run	γ	λ	δ	C1	C2	C3
Run 1	0.99	0.95	0.5	30	0.05	0.2
Run 2	0.99	0.97	0.5	30	0.1	0.2
Run 3	0.99	0.97	0.5	30	0.1	0.03
Run 4	0.99	0.99	0.5	30	0.3	0.2
Run 5	0.99	0.95	0.5	30	0.05	0.03
Run 6	0.99	0.99	0.5	30	0.15	0.2

Table 3. TRPO Run Parameters

5.2.1. Average Reward and Episode Length

As shown in Figures 2, TRPO's average reward was highly variable, with most runs yielding rewards between 2.5 and 4.5, and even the best cases only reaching around 8. This suggests that TRPO struggled to achieve high-reward states, likely due to overly conservative updates enforced by the strict KL divergence constraint and the brittleness of second-order optimization in high-dimensional, vision-based tasks. The stringent trust region limits seem to have prevented effective exploration, causing the policy to plateau at suboptimal levels. Furthermore, the mean episode length of about 68 steps indicates that the agent frequently terminated episodes prematurely, reflecting poor control and unstable behavior.

5.2.2. Entropy

Similarly, many of the runs had high entropy for TRPO. For a categorical distribution over four actions, the maximum entropy is log(4)=1.386, representing complete uncertainty with a uniform distribution. In our experiments, even in our best-performing TRPO run, the policy entropy only dropped to about 0.89, indicating that the agent never fully committed to a deterministic strategy. This persistent high entropy suggests that the policy remained excessively stochastic, likely due to weak advantage signals or overly conservative trust region constraints that hindered effective exploitation of learned behavior.

5.2.3. Sample Efficiency

In our experiments, TRPO's sample efficiency was consistently very low, with reward per environment step measured at less than 0.0013, compared to PPO's approximately 0.01. Even when training was extended beyond 3000 epochs, the interdependencies among hyperparameters—such as the strict KL divergence threshold, conjugate gradient iterations, and damping factors—prevented any significant improvement.

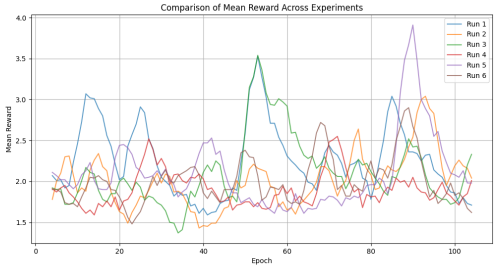
5.3. Comparison with TRPO

Metric	TRPO	PPO
Highest Reward	~ 8	~ 30
Average Reward	~ 4.5	~ 25
Mean Episode Length	68	150
Epochs to Convergence	Very slow (> 3000)	Moderate (< 1000)
Sample Efficiency	Very low (< 0.0013)	Moderate (~ 0.01)
Policy Entropy	1.386* (many runs) / 0.89 (late)	0.56

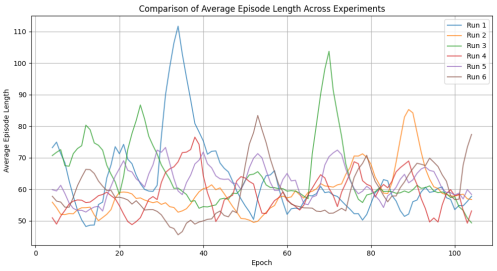
Table 4. Comparison of TRPO and PPO performance metrics in Breakout-v5.

In our experiments, the TRPO agent consistently underperformed relative to PPO. It struggled to surpass an average reward of 7 and often got stuck in local minima early in training. We attribute this to the following factors:

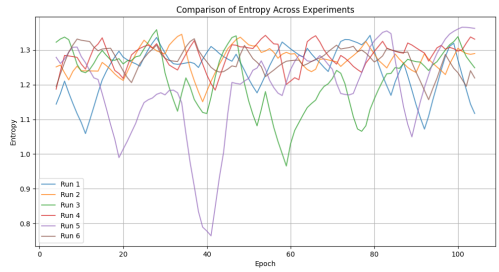
1. **Reward Metrics (Highest and Average Reward):** PPO achieves a peak reward of around 30 with an average of approximately 25, whereas TRPO's best runs only reach about 8 with an average of 4.5. This significant disparity indicates that TRPO's overly conservative updates—stemming from its strict KL-divergence constraint—severely limit its ability to explore and exploit high-reward states, resulting in persistently suboptimal performance.
2. **Episode Length and Convergence Speed:** PPO's policies yield mean episode lengths of roughly 150 steps and converge in fewer than 1000 epochs, while TRPO's policies terminate around 68 steps on average and require



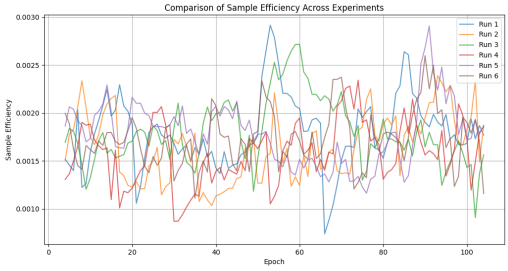
(a) Average Reward



(b) Average Episode Length



(c) Entropy



(d) Sample Efficiency

Figure 2. Training metrics for TRPO under different hyperparameter configurations.

over 3000 epochs to converge. The shorter episodes in TRPO suggest that the agent frequently loses control or fails to maintain effective strategies, and the prolonged convergence period reflects both the computational burden and inefficiency of its second-order optimization methods.

3. **Sample Efficiency:** With PPO achieving around 0.01 reward per environment step compared to TRPO's less than 0.0013, the stark difference in sample efficiency underscores how TRPO's strict update constraints and interdependent hyperparameters waste valuable samples. This inefficiency makes TRPO particularly impractical in data-scarce, high-dimensional settings.
4. **Policy Entropy:** For a categorical distribution over four actions, the maximum entropy is approximately 1.386, indicating complete uncertainty. In our experiments, even in the best-performing TRPO runs, the policy entropy only dropped to about 0.89, demonstrating that the policy remained excessively stochastic and failed to converge toward a more deterministic strategy. In contrast, PPO reduced its entropy to roughly 0.56, reflecting more confident, exploitative behavior.
5. **KL Constraint Sensitivity:** TRPO's reliance on a strict KL-divergence constraint necessitates very precise hyperparameter tuning—particularly the KL threshold and the number of conjugate gradient iterations. When these parameters are not optimally configured, the resulting updates become overly conservative, stifling exploration and causing the policy to plateau prematurely.
6. **Computational Overhead:** The second-order optimization approach in TRPO, which includes conjugate gradient and iterative line search procedures, incurs significant computational cost. This not only slows down convergence but also complicates experimental tuning, as evidenced by our encounters with out-of-memory errors during longer episodes—problems that are not observed with the more streamlined PPO.
7. **Lower Flexibility:** Unlike TRPO, which rigidly enforces a trust region via a KL constraint, PPO's clipped surrogate objective provides a more flexible update mechanism. This heuristic constraint is better able to accommodate noisy gradients and varying reward signals, resulting in a more robust performance and easier hyperparameter tuning, which ultimately contributes to PPO's superior empirical performance.

Our results strongly support PPO as the more stable and efficient algorithm for training policies in high-dimensional environments like Breakout. While TRPO offers theoretical guarantees such as monotonic policy improvement, our findings and experiments with the Breakout environment show that there is no free lunch; TRPO's practical implementation is hindered by extreme sensitivity to hyperparameters, significant computational overhead, and difficulties in scaling to complex tasks. In contrast, PPO's simple yet effective clipping mechanism enables fast convergence and robust performance across a wide range of hyperparameter settings, making it a more practical choice for this application.

6. Limitations and Assumptions

Despite our comprehensive evaluation, several limitations and assumptions in our approach may have hindered performance. First, our implementation relies on a standard preprocessing pipeline (frame skipping, grayscale conversion, downsampling, normalization, and frame stacking) which may not be optimal for TRPO's sensitivity to input representations. Additionally, the hyperparameter search space—although extensive with over 150 configurations—may still have overlooked certain critical settings or interactions that could potentially improve TRPO's performance. Our choice of a shared CNN architecture and training protocol, while effective for PPO, might inadvertently favor one algorithm over the other. Furthermore, our experiments are based solely on the Breakout environment, and while it is a challenging high-dimensional task, the results may not fully generalize to other domains. The inherent randomness in on-policy methods, even with fixed seeds, can introduce variability that may mask subtle improvements in TRPO's performance. Moreover, we explored avenues such as refining the reward structure and adding additional penalizing terms to potentially accelerate convergence; however, these modifications were not investigated in sufficient depth. It is also noteworthy that another group, which applied TRPO to a cataract detection problem formulated as an MDP, achieved remarkable success—likely due to differences in their underlying network architecture and problem formulation—indicating that TRPO's efficacy can vary significantly depending on the specific task and design choices. Overall, these factors underscore the complexity of achieving a fair comparison and suggest that further investigation is warranted to isolate the impact of these design choices.

7. conclusion

In conclusion, our experiments in the Breakout-v5 environment demonstrate that while TRPO offers strong theoretical guarantees such as monotonic policy improvement, its practical performance is severely limited by extreme sensitivity to hyperparameter settings, excessive computational overhead, and difficulties in scaling to high-dimensional, vision-based tasks. PPO, with its simpler and more flexible clipped surrogate objective, consistently outperformed TRPO in terms of highest and average rewards, episode length, sample efficiency, and convergence speed. These findings indicate that despite TRPO's appealing mathematical properties, the practical challenges—stemming from its strict KL divergence constraint and second-order optimization methods—render it less competitive compared to PPO in our experimental setting.

8. Future Work

Future research should further explore modifications to the reward structure, such as incorporating additional penalizing terms or reward shaping techniques, to potentially accelerate convergence for TRPO. Investigating alternative network architectures and preprocessing pipelines tailored to TRPO's optimization dynamics may also enhance its performance. Moreover, exploring hybrid approaches that integrate the theoretical strengths of TRPO with the empirical robustness of PPO could yield promising results. It is noteworthy that another group achieved significant success with TRPO on a cataract detection problem formulated as an MDP, likely due to differences in network design and problem structure, suggesting that TRPO's efficacy may improve in tasks with different characteristics. Further studies across a diverse set of domains will be essential to fully understand and leverage the potential of trust region methods.

References

[1] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. 3

[2] Richard H Byrd, Jean Charles Gilbert, and Jorge Nocedal. Trust region methods based on interior point techniques for nonlinear programming. *Mathematical Programming*, 89(1): 149–185, 2000. 2

[3] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on ppo and trpo, 2020. 1, 2

[4] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. *International Conference on Machine Learning (ICML)*, 2:267–274, 2002. 2

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 2, 3

[6] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016. 2

[7] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897, 2015. 1, 2, 3, 4

[8] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015. 2, 3, 4

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017. 1, 2, 4

[10] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. 1

[11] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. 1, 2

[12] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems (NeurIPS)*, 12:1057–1063, 2000. 2

[13] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. 4

[14] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992. 2